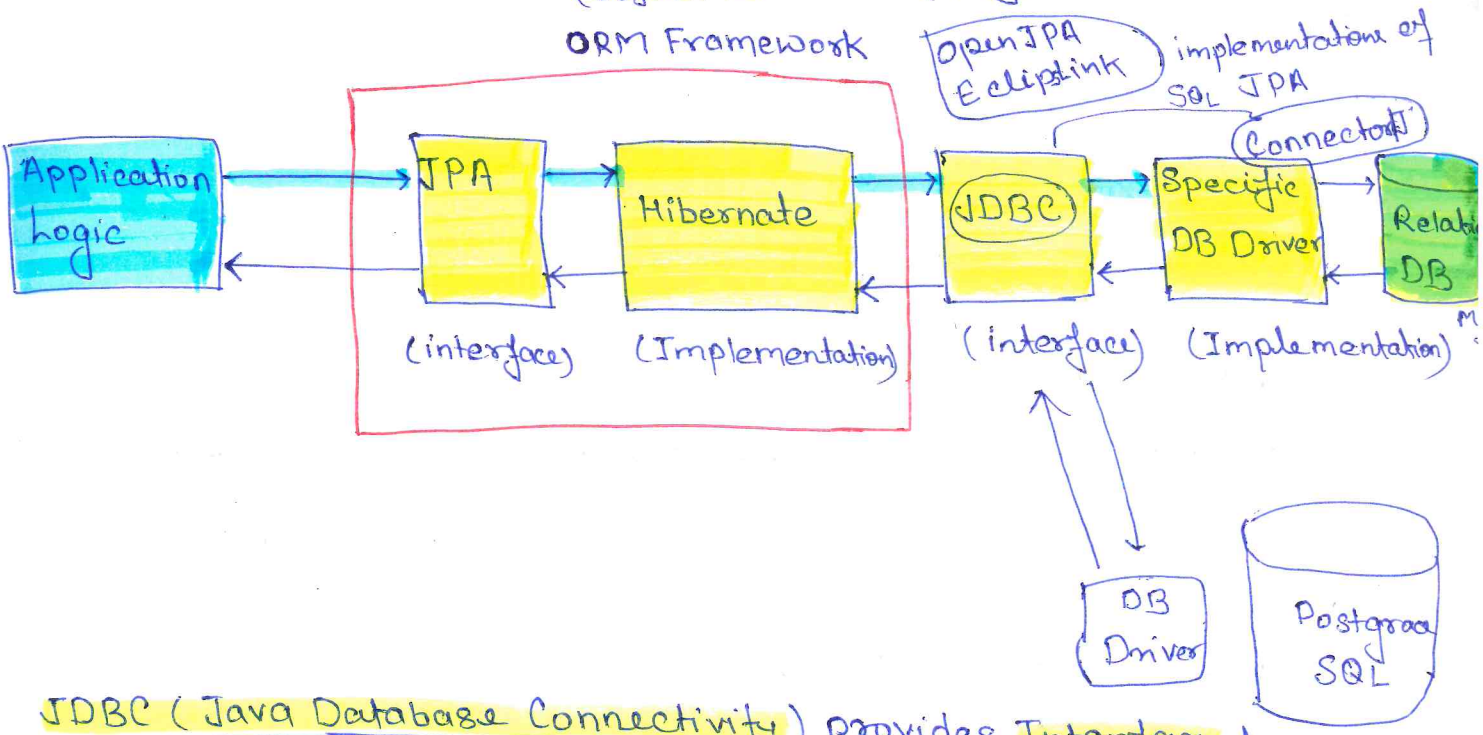
Spring Boot JPA

Bridge b/w Java Object - RDBMS
(Object Relation Mapping)
ORM Framework



JDBC (Java Database Connectivity) provides Interface to:

- Make Connection with DB
- Query DB
- and process the result.

Actual implementation is provided by Specific DB Drivers.

MySQL

- Driver : Connector/J
- Class : com.mysql.cj.Jdbc.Driver

PostgreSQL

- Driver : PostgreSQL JDBC Driver
- Class : org.postgresql.Driver.

Concept && Coding

Using JDBC without Springboot

```
Public class DatabaseConnection{
    Public Connection getConnection(){
      try{
        // H2 Driver loading
        Class.forNam("org.h2.Driver");

        // Establish connection with DB
        return Drivermanager.getConnection("JDBC:h2:mem:userDB",
                                           "root", password:"" )
      }
      catch(ClassNotFoundException|SQLException e){
          // handle exception
      }
      return null;
    }
}
```

```
Public class UserDAO{
    Public void createUserTable(){
      try{
        Connection connection = new DatabaseConnection().getConnection
        Statement statementQuery = connection.createStatement();
        String sql = "Create Table users(user-id INT AUTO_INCREME
                      PRIMARY KEY, user-name VARCHAR
                      (100), age INT;
        statementQuery.executeUpdate(sql);
      }
      catch(SQLException e){
      }
```

Concept && Coding

```
Public void createUser(String userName, int userAge){
try{
Connection connection = new DatabaseConnection().getConnection();
String SQLQuery = "INSERT INTO User(User_name, age) values(?,?)"
PreparedStatement preparedQuery = connection.prepareStatement
                                                    (SqlQuery)
preparedQuery.setString(1, userName);
preparedQuery.stInt (2, userAge);
preparedQuery.executeUpdate();

}
catch(SQLException e){

}
}
```

```
Public void readUsers(){
try{
Connection connection = new DatabaseConnection().getConnection();
String sqlQuery = "SELECT * FROM users";
PreparedStatement preparedQuery = connection.preparedStatement
ResultSet output = preparedQuery.executeQuery();(SqlQuery);
While(output.next()){
String userDetails = output.getInt("user-id") +
                    ":" + output.getString("user-name") +
                    ":" + output.getInt("age");
        S.o.p(userDetails);
}
}
Catch(SQLException e){

}
}
```

Concept && Coding

If we see above example.

- Connection
- Statement
- prepared Statement
- ResultSet etc.

All are interfaces which JDBC provide and each specific driver provide the implementation for it

But there are so much of BOILERCODE present like:

- Driver class loading
- DB Connection Making
- Exception Handling
- Closing of DB Connection and other objects like statement etc.
- Manual handling of DB Connection Pool.

Ect..

Using JDBC with Springboot

1 : we add Two dependency in pom.xml

(a) Spring-boot-Starter-jdbc

(b) MySQL Connecter J

Springboot provides JDBC Template class, which helps to remove all the boiler code.

```
class User {
    int userID;
    String name;
    int age;
    // getter
```

Concept && Coding

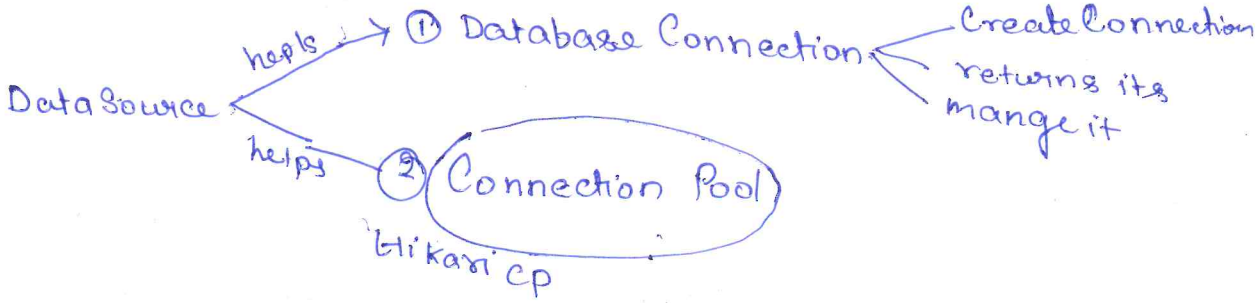
2 @Repository — 1 @Component — 2 Granulable level of SQL Exception Return

```
Public class UserRepository{

    @Autowired
    JdbcTemplate jdbcTemplate;

    Public void CreateTable(){
        JdbTemplate. execute ("Creat Table Quer")
    }

    Public void insertUser (String name, int age){
        String insertQuery = "Insert data into Table Query"
        JdbcTemplate . update (insertQuery, name;age);
    }

    Public List<User> getUser ( ){
        String selectQuery = "Select Table dat Query";
        return jdbcTemplete . query (selectQuery, (rs, rowNum)->{
            User user = new User();
            User. SetUserId (rs.getInt ("user_id"));
            User. SetUserName (rs. getString ("user-name"));
            User. SetAge (rs. getInt ("age"));
            retun user;
        });
    }
}
```

Concept && Coding

application.properties: (3)

```
spring.datasource.url:
spring.datasource.driver-class-name = com.mysql.cj.jdbc.Driver
                    username:
                    password:
```



Connection Pool - It is way to manage a group of reusable connection to database.

Driver class Loading:-

JdbcTemplate load it at the time of application startup in DriverMamager class.

DB Connection Making:-

JdbcTemplate takes care of it, whenever we execute any query.

Exception Handling:-

In Plain JDBC, we get very abstracted 'SQLException' but in JdbcTemplate, we granular error like. DuplicateKeyException, QueryTimeoutException etc. (defined in org.springboot framework.dao package).

Concept && Coding

Closing of the DB connection and other resources -

When we invoke update or query method, after success or failure of the operation, JdbcTemplate take care of either closing or return the connection to pool itself.

- Manual handling of DB Connection Pool -

Springboot provides default jdbc connection pool i.e 'HikariCP' with Min and Max pool size of 10.

And we can change the configuration in "application properties".

```
spring.datasource.hikari.maximum-pool-size = 10;
spring.datasource.hikari.minimum-idle = 5;
```

We can also configure different jdbc connection pool if want like below:

```
@Configuration
Public class AppConfig {

@Bean
Public DataSource dataSource () {
    HikariDataSource dataSoure = new HikariDataSource()
    dataSoure.setDriverClassName("org.h2.Driver");
    dataSource.setJDbcUrl( )
    dataSource.setUsername( );
    dataSource.SetPassword( );
    return dataSource;
}
}
```

Concept && Coding

update (String sql, Object args)

Insert
Update
Delete

Custom Interceptors in Springboot

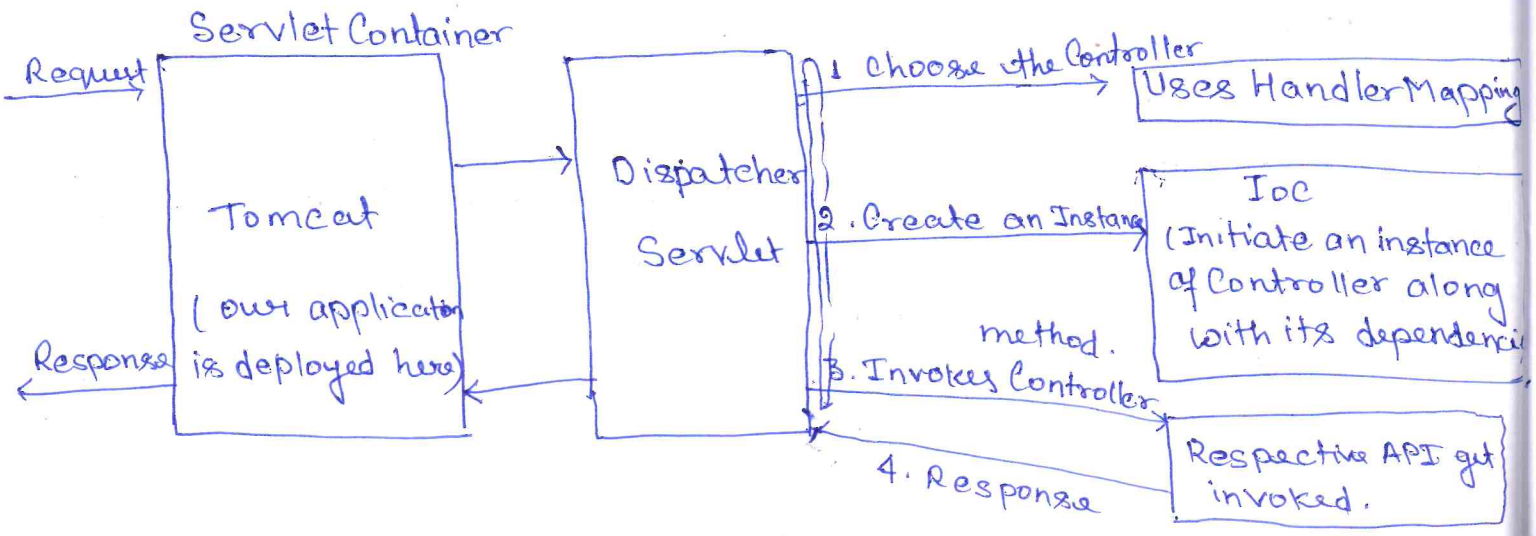
Interceptor :-

It's a mediator, which get invoked before and after your actual code.

Write our own custom interceptors:

- Springboot caching
- Springboot logging
- Springboot Authentication etc.

① Custom Interceptor for Requests before even reaching to specific Controller class.



Concept && Coding